

Distributed Systems - 12_11

Consensus in Eventually Synchronous Systems

What happens when we consider a system that is ES or A system? If the system is async, we can't do much. We have a theorem::

- (FLP): Consider an async system composed by n processes. If $f > 0$ then consensus is unsolvable in such system.

Even if we have a single failure that may or may not happen, we can't solve consensus.

The proof shows that for any algorithm there exists a run in which you either:

- violate a safety property
- violate liveness

Since we cannot solve it in the async, we try to solve it in eventually sync systems. We'll see that eventually sync is good enough for a lot of cases.

Paxos provides a solution to consensus in eventually sync systems with crash failures. It always satisfies the safety properties, like integrity and agreement. Since Paxos is working in ev. sync systems, we could have moments in which the system goes above and below a synchrony bound. However, we'll have a time T^* after which we'll always be below the synchrony bound.

Paxos ensures liveness when we are after T^* , so in the synchrony period, guaranteeing that Paxos terminates.

How does Paxos guarantee the safety? When the system becomes async, he stops the decisions, at least some of them.

So Paxos assumes a majority of correct processes, as well as that we are in an eventually synchronous system.

With Paxos, we introduce roles. A process, in the system, can execute one or multiple of this roles. We could have a proposer role, in which the process proposes a value. We'll have that the set of proposers will be limited.

We also have the acceptors, processes that run an algorithm that choose the consensus value. A third role is given by learner, a learner is to learn the value of the consensus, he will see the result of the terminated consensus, with the decided

value.

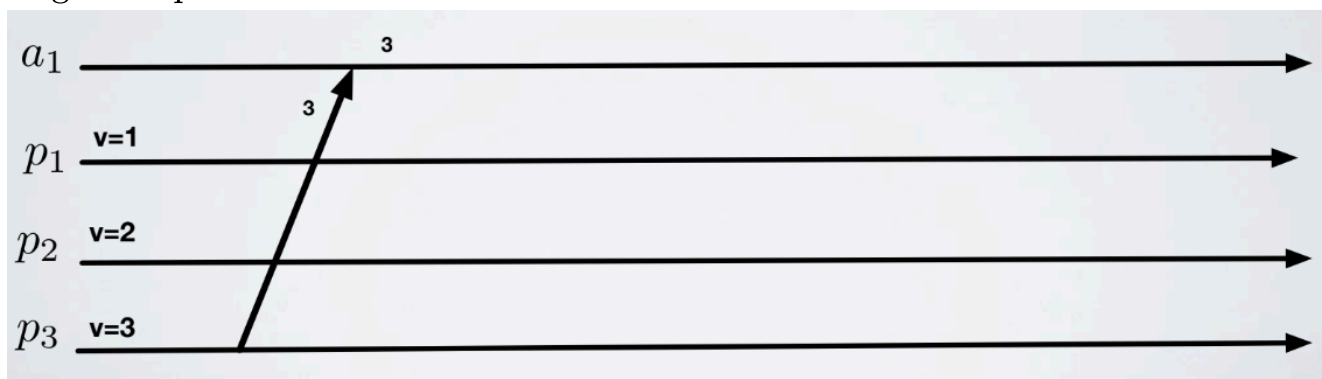
For example, p_1 could run a thread for the proposer, a thread for the acceptor, and another one for the learner, so he can do whatever.

The algorithm will work for any amount of failures on proposer, any minus one for the learner, and the majority of correct processes for the acceptors.

A consensus protocol has to decide a value based on agreement. Before doing that, we also have to choose the value. Choosing the value is the instant in which our algorithm has already decided the algorithm but does not know it. In uniform consensus, we have p_1 doing the impose, receiving the ACKs, and then deciding. There is a moment in which the process choose a value (when someone is able to communicate with everyone else in uniform), and then a moment in which we decide. We have to define the exact moment in which the value is chosen.

In order to satisfy the consensus, only a value that has been chosen has to be decided. Also, when there is this chosen moment, only one value can be chosen. *If two processes choose two different values, there is no agreement.*

What is the easiest way to try to solve consensus? It's to create a system with a single acceptor.



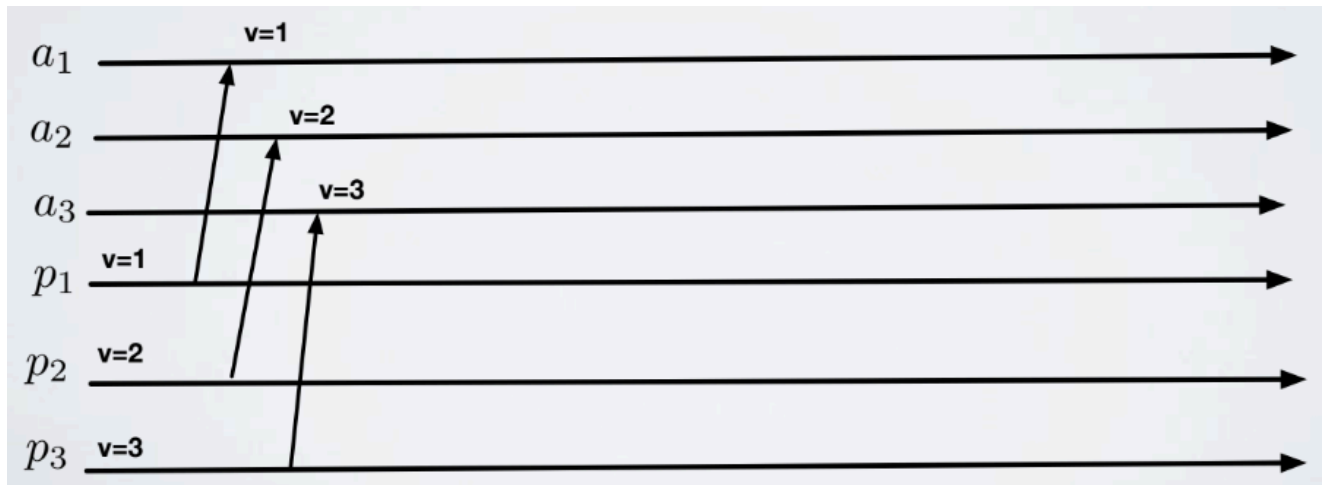
In the example we have three proposers, p_1, p_2, p_3 , and the message that arrives before to a_1 (acceptor) wins.

What happens, however, if the acceptor fails? We have to increase the number of acceptors, if we want to tolerate a failure we need three acceptors. When is the value chosen in this case? We could have the majority, so when a majority accepts a value, then it's chosen.

So the proposer sends a value to every acceptor, like p_1 sending 1, and when a_1 and a_2 (for example) accept this value, then it's chosen. It's acceptable as if a_3 receives, for example, $v = 3$ from p_3 first, it will not be a problem, as $v = 1$ has already been accepted. Incoming v to p_1 and p_2 will be rejected. This strategy has some problems: we know that the acceptor has to accept the first value that he receives. If we accept a value, and not other ones, then will have to reject new

proposals.

But then, we could have something like this:



We'll have a_1 accept 1, a_2 2 and a_3 3. When $v = 1$ will reach p_2 , then it will reject this, and so on, so we will never reach agreement. We have a deadlock.

How do we actually solve this problem? We have to change the fact that we accept a single proposal, in fact with some of the proposals a process will change idea. So an acceptor should accept multiple proposals.

We will keep track of different proposals by assigning them an id, that is unique. A proposal will be $\langle \text{round number}, \text{value} \rangle$. When the proposer sends a value to the acceptor, he could send something like $(r_n, p = 1)$, and another could send $(r'_n, p = 2)$.

How do we guarantee that different processes pick different round numbers?

We could use modulo, having something like this:

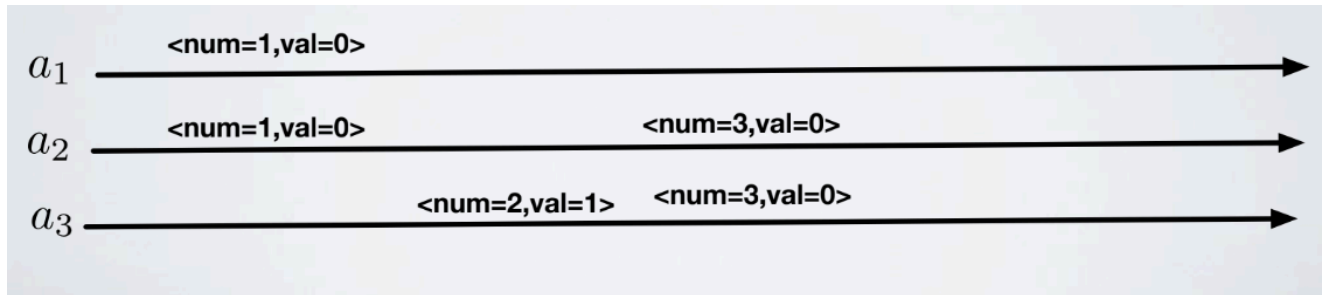
p_0	0	3	6	9
p_1	1	4	7	10
p_2	2	5	8	11

A property that we have to enforce is that if at a certain time we create a majority on value 0, then in the future we create another majority, then it has to be on value 0 as well.

So, if a proposal with value v is chosen, then every higher-numbered proposal (future proposal) that is chosen has value v .

From now on, to implement some properties, we'll create a property that implies another property, and so on.

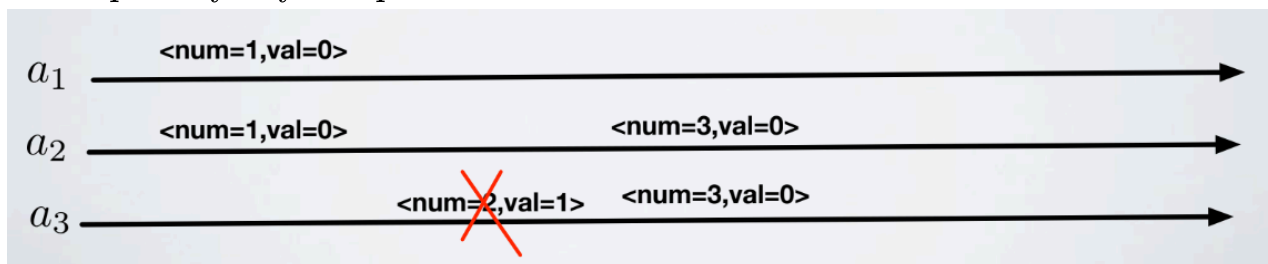
We want to avoid a situation like this:



Here another majority is accepted.

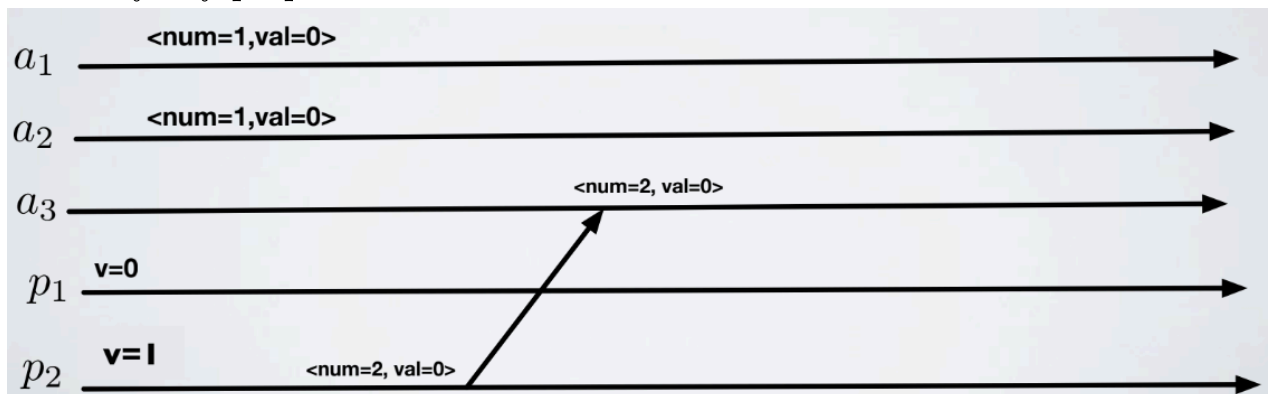
Let's take a look at property $p2a$:

- (p2a) if a proposal with value v is chosen, every higher-numbered proposal that is accepted by any acceptor has value v .



The best way to do this, is to enforce the proposer to not propose something we do not like, so we have property $p2b$:

- (p2b): If a proposal with value v is chosen, every higher-numbered proposal issued by any proposer has value v .



Now, how do we guarantee this? Before sending the proposal, we ask the acceptors if they accepted proposals, and when he'll receive a majority of messages, he picks the value with highest round number, and modify his proposal. So we'll have a query phase, from which we collect answers and pick values, sending back as our proposal.

Thus, we'll have property p2c.

P2c: For any $\mathbf{v}$ and $\mathbf{n}$, if a proposal with value $\mathbf{v}$ and round number $\mathbf{n}$ is issued, then there is a set S consisting of a majority of acceptors such that either:

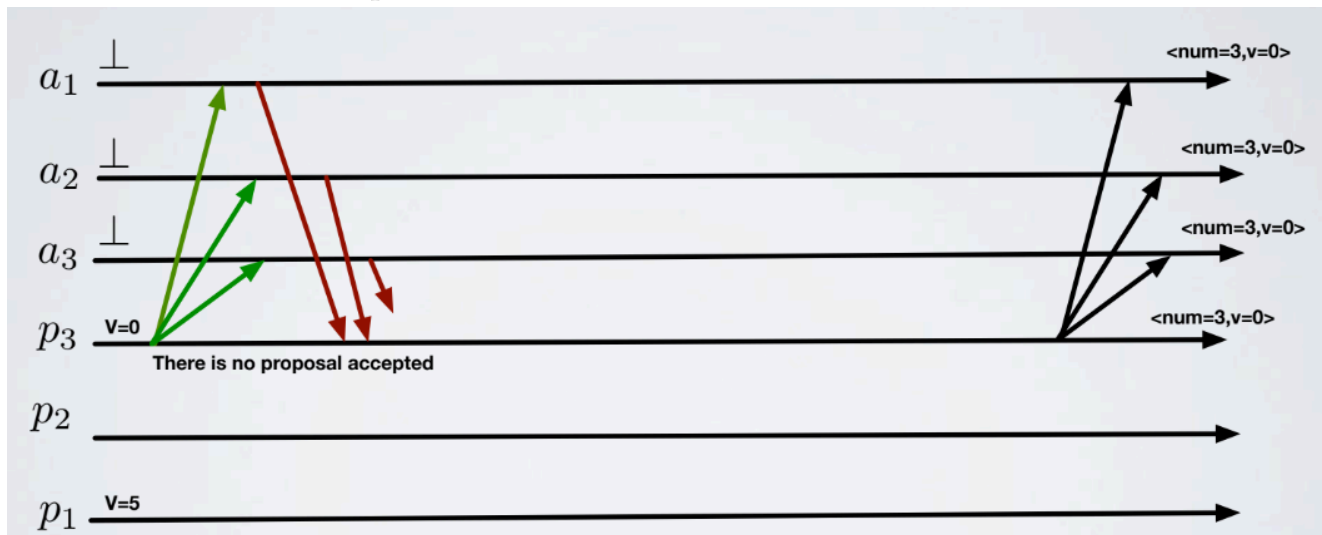
(a) no acceptor in S has accepted any proposal round numbered less than $\mathbf{n}$.

(b) $\mathbf{v}$ is the value of the highest-numbered proposal among all proposals round numbered less than $\mathbf{n}$ accepted by the acceptors in S .

P2c $\rightarrow$ P2b

Let's say that in the system happens before us, such that a_1 and a_2 has accepted $(1, 0)$. What a new proposer does is a prepare phase, and so he will send a message to each acceptor asking for the value they've accepted. Once we reach an answer from the majority of acceptors, we take a look at the answers and take the one with the highest round number. We then change our proposal, and send it to other acceptors.

Let's take a look at a specific case:



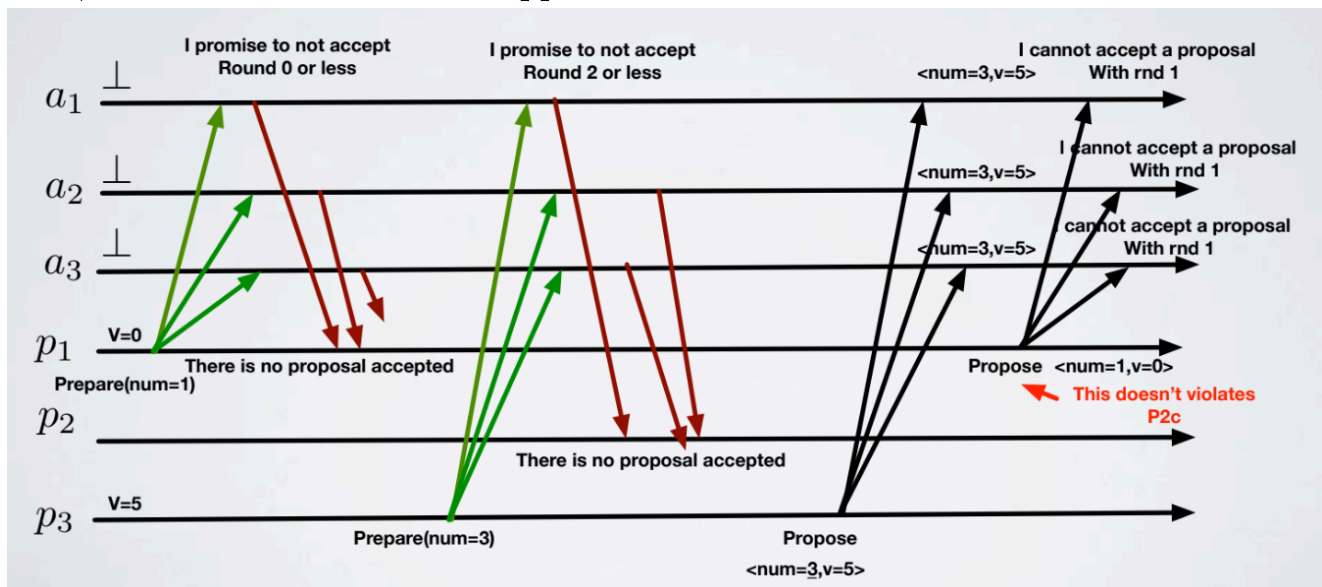
After receiving an answer saying that no acceptor accepted anything, we propose a new value. Between the receival of answers and the proposer, we could have a really long time, we could have an unbounded delay.

It's possible that, in the meanwhile, another process could start a preparation phase, and see that no one has accepted anything. So I could propose a new value, $\langle 1, 5 \rangle$, and send it to the acceptors. Then p_3 proposes the value, and we would be creating first a majority for $\langle 1, 5 \rangle$, and in the future a new one for $\langle 3, 0 \rangle$, thus breaking the algorithm.

How do we fix this execution? The problem is that p_1 is able to start round number 1, when round number 3 has already started. The majority of the system was already in round number 3. We'll put in the prepare message that a prepare has a round number. So the acceptor will remember that he is in round number 3, and from now on if an acceptor receives a proposal for round number 1 they will reject as they are already in round number 3 (and will not accept prepare with round number of 2 and less).

There is a specific instant in which we kill the prepare for a specific round number, when the majority of acceptors have memorized a specific round number.

Now, a situation like this can happen:



So the Paxos protocol has two phases:

In phase 1, when a proposer wants to propose a value, he chooses a new round number n , and send a prepare request ($PREPARE, n$) to all acceptors.

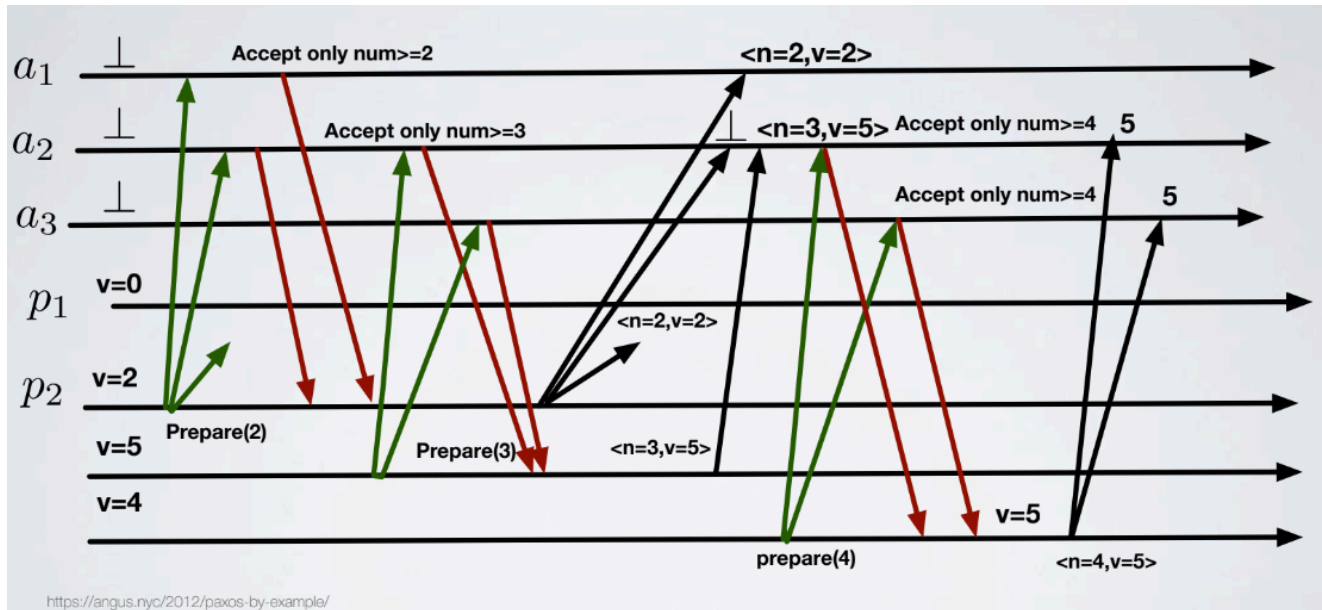
The acceptors that receive this request, with an $n > n'$ for any n' that has already been saved, will send an ACK. If $n < n'$, they will send a NACK. By responding with an ACK, we put (ACK, n, n', v') , or $(ACK, n, \perp, \perp)$ if it's the first proposal. Otherwise, if I'm at round number 10, and receive a proposal with round number 5, we answer with $(NACK, n')$.

In phase 2, if a proposer receives a majority of ACKs, then he collect the response with highest number, and proposes it.

If an acceptor receives receives an accept request ($ACCEPT, n, v$), it accepts the proposal unless it has already responded to a prepare request having a number ...

At this point we still did not decide, when do we do it? We'll use learners, such that acceptors will tell the learners the value they accepted, and when the learner sees that there is a majority of a certain value, he will learn that value.

Let's see an example of execution:



After p_3 receiving ACKs, is it possible that for $r = 2$ we reach a majority? No, as only a_1 could be convinced when p_2 proposes a value, a_2 and a_3 will reject as now they've memorized $r = 3$. At the end, we'll have that a_2 and a_3 (so the majority) will have accepted 5 as a value.

In the best case, how many messages do we need for Paxos to terminate? We need n for the prepare, n messages for the ACKs, and n for the proposal. In the worst case, how many messages do we need? They are unbounded, we do not know.